Group 10: Laboratory 2

Spatial Filtering and Thresholding

Keegan Rolls (202004149)
Nicholas Philpott (202018248)

ECE 7410: Image Processing

Memorial University of Newfoundland

July 1st, 2025

Spatial Filtering

All code contained within this section can be seen in it's entirety in Appendix A or in the submitted file `spatialFiltering.m`

Smoothing

1. *Download the test images (`img1.png` and `img2.png`) from Brightspace under Lab 2.*
2. *Read `img1.png`, convert it to a grayscale image, and display.*

   See Figure 1.a Below.
3. *Develop and display a function that will perform spatial filtering (i.e., convolution) on an $M \times N$ pixel image using a general $m \times n$ kernel. The function should take an image and a kernel as inputs and return the filtered image. Do **NOT** use internal functions for this (e.g., MATLAB's **imfilter** function). (**hint:** zero padding the edges of the image is required).*

```
%% Q1.3 Function: Custom Spatial Filtering
function output = my_filter2D(img, kernel)
    % Convert inputs to double for calculations
    img = double(img);
    kernel = double(kernel);

    % Get image and kernel dimensions
    [M, N] = size(img);
    [m, n] = size(kernel);

    % Calculate padding needed (zero padding)
    pad_m = floor(m / 2);
    pad_n = floor(n / 2);

    % Zero pad the image
    padded_image = zeros(M + 2 * pad_m, N + 2 * pad_n);
    padded_image(pad_m + 1:M + pad_m, pad_n + 1:N + pad_n) = img;

    % Initialize output image
    output = zeros(M, N);

    % Perform spatial filtering
    for i = 1:M
        for j = 1:N
            % Extract region from padded image
            region = padded_image(i:i + m - 1, j:j + n - 1);
            % Apply spatial filtering: (element-wise multiplication and sum)
            output(i, j) = sum(sum(region .* kernel));
        end
    end

    % Convert back to uint8 and clamp values to [0, 255]
    output = uint8(max(0, min(255, output)));
end
```

4. *Filter `img1.png` with each of the five filters below (Kernel 1 to Kernel 5) using your function and display the outputs:*
   1. `Kernel_1` $= \left(\frac{1}{9}\right) *$`ones`$(0)$
   2. `Kernel_2` $= \left(\frac{1}{49}\right) *$`ones`$(7)$
   3. `Kernel_3` $=$ `fspecial`$(\text{'average'}, [7, 7])$

4. `Kernel_4` = `fspecial`('gaussian', [3, 3], 0.5)
5. `Kernel_5` = `fspecial`('gaussian', [7, 7], 1.2)

```
%% Q1.4: Apply spatial filters
% Define Kernels
kernel1 = (1/9) *ones(3);
kernel2 = (1/49)* ones(7);
kernel3 = fspecial('average', [7, 7]);
kernel4 = fspecial('gaussian', [3, 3], 0.5);
kernel5 = fspecial('gaussian', [7, 7], 1.2);

% Apply spatial filters
filtered1 = my_filter2D(img1_gray, kernel1);
filtered2 = my_filter2D(img1_gray, kernel2);
filtered3 = my_filter2D(img1_gray, kernel3);
filtered4 = my_filter2D(img1_gray, kernel4);
filtered5 = my_filter2D(img1_gray, kernel5);
```

See Figures 1.b–f Below

(a) `img1.png` converted to greyscale

(b) `img1.png` with 'Kernel_1' applied

(c) `img1.png` with 'Kernel_2' applied

(d) `img1.png` with 'Kernel_3' applied

(e) `img1.png` with 'Kernel_4' applied

(f) `img1.png` with 'Kernel_5' applied

Figure 1: Smoothing filters applied to `img1.png`

5. *Develop and display a function that performs median filtering. A median filter will replace the pixel intensity with the median intensity of the pixels overlapped by the kernel. The function should take the image and the size of the kernel as inputs. (**hint:** zero padding the edges of the image is required).*

```
%% Q1.5 Function: Custom Median Filtering
function output = my_median_filter(img, kernel_size)
    % Convert to double for calculations
    img = double(img);

    % Get image dimensions
    [M, N] = size(img);

    % Calculate padding needed (zero padding)
    pad = floor(kernel_size / 2);

    % Zero pad the image manually
    padded_image = zeros(M + 2 * pad, N + 2 * pad);
    padded_image(pad + 1:M + pad, pad + 1:N + pad) = img;

    % Initialize output image
    output = zeros(M, N);

    % Perform median filtering
    for i = 1:M
        for j = 1:N
            % Extract region from padded image
            region = padded_image(i:i + kernel_size - 1, j:j + kernel_size - 1);
            % Calculate median of the region
            output(i, j) = median(region(:));
        end
    end

    % Convert back to uint8
    output = uint8(output);
end
```

6. *Apply the following two median filters to* ***img1.png*** *and display the outputs:*
    1. Median filter with kernel size 3×3
    2. Median filter with kernel size 5×5

```
%% Q1.6 Apply custom median filters
filtered_median_3 = my_median_filter(img1_gray, 3);
filtered_median_5 = my_median_filter(img1_gray, 5);
```

See Figures 2.a and 2.b below.

(a) `img1.png` with median filter 3×3 applied

(b) `img1.png` with median filter 5×5 applied

Figure 2: Median filters applied to `img1.png`

7. *Compare and discuss the smoothing effects obtained by `Kernel_1` to `Kernel_5` and the two median filters.*

   Kernel 1 will slightly blur the image, and lose some small details in the process, such as the strands of her hair. Kernels 2 and 3 are the same, except that 3 uses the fspecial built-in MATLAB function, and 2 uses the direct kernel calculation. In 4, if we compare it to 1 and 2, we notice that the strands of the woman's hair are blurry, but her face maintains more detail than these images. In 5, the woman becomes more blurry as the kernel size is increased from 3x3 to 7x7, but she still has more details in her face and her hair when compared to 2 and has less detail when compared to 1. Median filter 1 is similar to a blurring kernel 1, except with subtle intensity differences, and it contains less detail than the Gaussian blur 3. Median filter 2 has less detail than the 3x3 blurring kernels, but has more details than the 7x7 kernels.

8. *Discuss the effect the smoothing kernel size has on the output images in your experiments.*

   From the images, we can see that increasing the kernel size from 3x3 to 7x7 increases the amount of blurring in the overall image. In 1 and 2 we notice that as the size of the kernel increases, it loses more detail. In 4 and 5 we notice that the Gaussian operation, increasing the kernel, loses some detail, but less than the average operation. In the median filter, we notice the same thing, that it loses some detail when it is blurred.

Sharpening

1. *Use the function you developed in* ***Smoothing: Step 3*** *and apply the following three sharpening filters (*`Kernel_6` *to* `Kernel_8`*) to* `img2.png`*.*

   1. `Kernel_6` $= \begin{bmatrix} -1 & 0 & 1 \\ -2 & 0 & -2 \\ -1 & 0 & 1 \end{bmatrix}$
   2. `Kernel_7` $= \begin{bmatrix} -1 & -2 & -1 \\ 0 & 0 & 0 \\ 1 & 2 & 1 \end{bmatrix}$
   3. `Kernel_8` = `fspecial`('log', 3)

```
%% Q2: Sharpening and Edge Detection
% Read img2 and convert to grayscale
img2 = imread('./L2/img2.png');
img2_gray = rgb2gray(img2);

% Define sharpening kernels
kernel6 = [-1, 0, 1; -2, 0, 2; -1, 0, 1];
kernel7 = [-1, -2, -1; 0, 0, 0; 1, 2, 1];
kernel8 = fspecial('log', 3);

% Apply sharpening filters using the custom function
filtered6 = my_filter2D(img2_gray, kernel6);
filtered7 = my_filter2D(img2_gray, kernel7);
filtered8 = my_filter2D(img2_gray, kernel8);

% Apply filters to sharpen the original image
% Convert to double for arithmetic operations and clamp results
sharpened_img6 = uint8(max(0, min(255, double(img2_gray) + double(filtered6))));
sharpened_img7 = uint8(max(0, min(255, double(img2_gray) + double(filtered7))));
sharpened_img8 = uint8(max(0, min(255, double(img2_gray) - double(filtered8))));
```

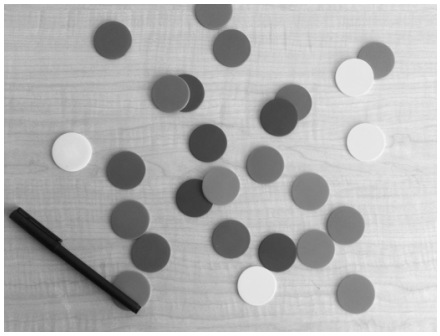
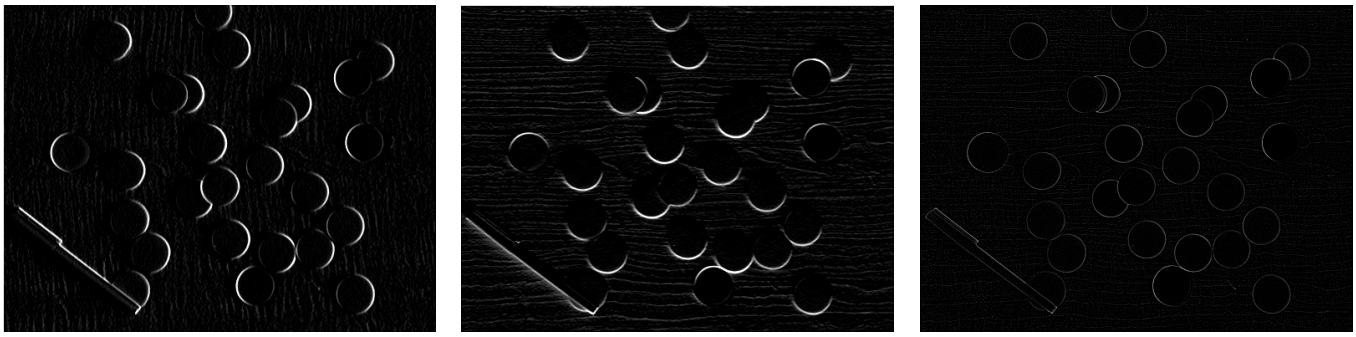


Figure 3: `img2.png` converted to greyscale

See Figures 4.a–f below.



(a) `img2.png` with `Kernel_6` applied (b) `img2.png` with `Kernel_7` applied (c) `img2.png` with `Kernel_8` applied

(d) `img2.png` sharpened with `Kernel_6` (e) `img2.png` sharpened with `Kernel_7` (f) `img2.png` sharpened with `Kernel_8`

Figure 4: Sharpening filters applied to `img2.png`

2. *Compare and discuss the outputs obtained by `Kernel_6` to `Kernel_8`*

   In Figure 4 above, `Kernel_6` to `Kernel_8` (4.a, b, c, respectively) are edge detection filters that highlight different features in the image. `Kernel_6` is a vertical edge detector (Sobel operator) that emphasizes vertical boundaries and transitions in the image, making vertical edges appear bright while horizontal features remain dark. `Kernel_7` is a horizontal edge detector that highlights horizontal boundaries and transitions, making horizontal edges prominent while vertical features are suppressed. `Kernel_8` uses the Laplacian of Gaussian filter which detects edges in all directions simultaneously, producing a more comprehensive edge map that captures both horizontal and vertical features. The sharpened images (4.d, e, f) show how adding these edge-enhanced versions back to the original image improves overall sharpness and detail definition.

Thresholding

The following equations are from the lab manual.

$$\omega_0(t) = \sum_{i=0}^{t-1} p(i) \quad \mu_0(t) = \sum_{i=0}^{t-1} ip(i)/\omega_0 \quad \omega_1(t) = \sum_{i=t}^{L-1} p(i) \quad \mu_1(t) = \sum_{i=t}^{L-1} ip(i)/\omega_1 \tag{1}$$

$$\sigma_b^2(t) = \omega_1\omega_2 \left[\mu_0 - \mu_1\right]^2 \tag{2}$$

1. *Download the test images (`img3.png` and `img4.png`) from Brightspace under Lab 2 and save them in your working directory.*

   See Figure 5.a and 5.b below.

2. *Develop and display code to calculate the class probabilities and class means, as well as the inter-class variance (equations 1 and 2) as a complete function to calculate the optimal threshold value for any given image.*

```
%% Q2: Function to calculate optimal threshold using Otsu's method
function optimal_threshold = calculate_optimal_threshold(img)
    % Convert to grayscale if needed
    if size(img, 3) == 3
        img = rgb2gray(img);
    end

    % Convert to double for calculations
    img = double(img);

    % Calculate histogram
    [counts, ~] = hist(img(:), 0:255);

    % Calculate probabilities
    p = counts / sum(counts);

    % Initialize variables
    L = 256; % Number of gray levels (0-255)
    max_variance = 0;
    optimal_threshold = 0;

    % Try all possible thresholds
    for t = 1:(L - 1)
        % Calculate class probabilities
        omega_0 = sum(p(1:t));
        omega_1 = sum(p(t + 1:L));

        % Skip if one class is empty
        if omega_0 == 0 || omega_1 == 0
            continue;
        end

        % Calculate class means
        mu_0 = sum((0:(t - 1)) .* p(1:t)) / omega_0;
        mu_1 = sum((t:(L - 1)) .* p(t + 1:L)) / omega_1;

        % Calculate inter-class variance
        sigma_b_squared = omega_0 * omega_1 * (mu_0 - mu_1) ^ 2;

        % Update optimal threshold if this variance is larger
        if sigma_b_squared > max_variance
            max_variance = sigma_b_squared;
            optimal_threshold = t - 1; % Convert to 0-255 range
        end
    end
end
```

3. *Calculate and report the optimal threshold values for* ***img3.png*** *and* ***img4.png*** *found using your code in* ***Thresholding: Step 2****.*

   Upon executing the file `thresholding.m` (See Appendix B) the optimal threshold values were printed to the console:

```
Optimal threshold for img3.png: 177
Optimal threshold for img4.png: 129
```

4. *Develop code to convert a grayscale image to a binary image using a given threshold value. Do* ***NOT*** *use internal functions for this (e.g., the MATLAB function* ***im2bw****).*

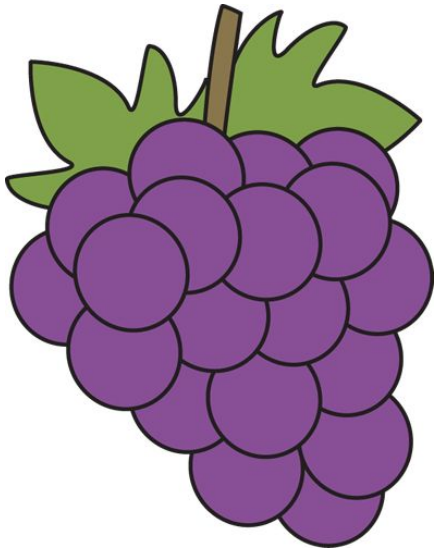
   Please note that `rgb2gray` is used only to ensure that the image is already a greyscale image.

```
%% Q4: Function to convert grayscale to binary using threshold
function binary_img = grayscale_to_binary(img, threshold)
    % Convert to grayscale if needed
    if size(img, 3) == 3
        img = rgb2gray(img);
    end

    % Convert to double for calculations
    img = double(img);

    % Get image dimensions
    [rows, cols] = size(img);

    % Initialize binary image
    binary_img = zeros(rows, cols);

    % Apply threshold manually using loops
    for i = 1:rows
        for j = 1:cols
            if img(i, j) > threshold
                binary_img(i, j) = 255; % White pixel
            else
                binary_img(i, j) = 0; % Black pixel
            end
        end
    end

    % Convert to uint8
    binary_img = uint8(binary_img);
end
```

5. *Use the optimal thresholds to generate and display binary image for* ***img3.png*** *and* ***img4.png****.*

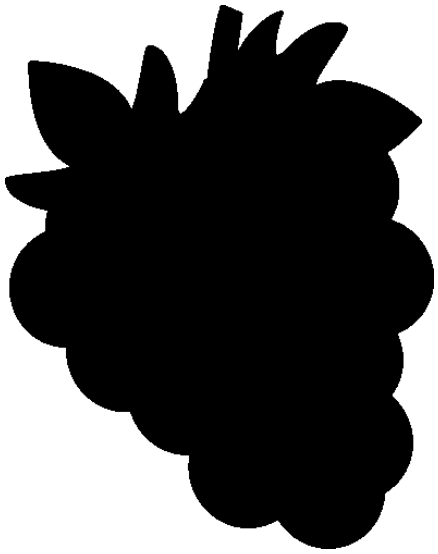
   See Figure 5.c and 5.d below.

```
% Convert to binary using optimal thresholds
binary_img3 = grayscale_to_binary(img3, threshold_img3);
binary_img4 = grayscale_to_binary(img4, threshold_img4);
```



(a) Original `img3.png`

(b) Original `img4.png`



(c) Binary `img3.png` with threshold = 177

(d) Binary `img4.png` with threshold = 129

Figure 5: Original images and their binary thresholded versions

Appendix A

This section contains the full code for both Spatial Filtering questions (Smoothing & Sharpening).

```
% ECE 7410 - Laboratory 2: Spatial Filtering

%% Q1.2: Read img1 and convert to grayscale
% Create output directory
if ~exist('./L2/outputs/01_smoothing', 'dir')
    mkdir('./L2/outputs/01_smoothing');
end

% Read image
img1 = imread('./L2/img1.png');
img1_gray = rgb2gray(img1);

% Save the grayscale image
imwrite(img1_gray, './L2/outputs/01_smoothing/img1_grey.png');

%% Q1.3 Function: Custom Spatial Filtering
function output = my_filter2D(img, kernel)
    % Convert inputs to double for calculations
    img = double(img);
    kernel = double(kernel);

    % Get image and kernel dimensions
    [M, N] = size(img);
    [m, n] = size(kernel);

    % Calculate padding needed (zero padding)
    pad_m = floor(m / 2);
    pad_n = floor(n / 2);

    % Zero pad the image
    padded_image = zeros(M + 2 * pad_m, N + 2 * pad_n);
    padded_image(pad_m + 1:M + pad_m, pad_n + 1:N + pad_n) = img;

    % Initialize output image
    output = zeros(M, N);

    % Perform spatial filtering
    for i = 1:M
        for j = 1:N
            % Extract region from padded image
            region = padded_image(i:i + m - 1, j:j + n - 1);
            % Apply spatial filtering: (element-wise multiplication and sum)
            output(i, j) = sum(sum(region .* kernel));
        end
    end

    % Convert back to uint8 and clamp values to [0, 255]
    output = uint8(max(0, min(255, output)));
end

%% Q1.4: Apply spatial filters
```

```
% Define Kernels
kernel1 = (1/9) * ones(3);
kernel2 = (1/49) * ones(7);
kernel3 = fspecial('average', [7, 7]);
kernel4 = fspecial('gaussian', [3, 3], 0.5);
kernel5 = fspecial('gaussian', [7, 7], 1.2);

% Apply spatial filters
filtered1 = my_filter2D(img1_gray, kernel1);
filtered2 = my_filter2D(img1_gray, kernel2);
filtered3 = my_filter2D(img1_gray, kernel3);
filtered4 = my_filter2D(img1_gray, kernel4);
filtered5 = my_filter2D(img1_gray, kernel5);

% Save filtered images
% Create kernel output directory
if ~exist('./L2/outputs/01_smoothing/04', 'dir')
    mkdir('./L2/outputs/01_smoothing/04');
end

imwrite(filtered1, './L2/outputs/01_smoothing/04/kernel1.png');
imwrite(filtered2, './L2/outputs/01_smoothing/04/kernel2.png');
imwrite(filtered3, './L2/outputs/01_smoothing/04/kernel3.png');
imwrite(filtered4, './L2/outputs/01_smoothing/04/kernel4.png');
imwrite(filtered5, './L2/outputs/01_smoothing/04/kernel5.png');

%% Q1.5 Function: Custom Median Filtering
function output = my_median_filter(img, kernel_size)
    % Convert to double for calculations
    img = double(img);

    % Get image dimensions
    [M, N] = size(img);

    % Calculate padding needed (zero padding)
    pad = floor(kernel_size / 2);

    % Zero pad the image manually
    padded_image = zeros(M + 2 * pad, N + 2 * pad);
    padded_image(pad + 1:M + pad, pad + 1:N + pad) = img;

    % Initialize output image
    output = zeros(M, N);

    % Perform median filtering
    for i = 1:M
        for j = 1:N
            % Extract region from padded image
            region = padded_image(i:i + kernel_size - 1, j:j + kernel_size - 1);
            % Calculate median of the region
            output(i, j) = median(region(:));
        end
    end
```

```
    % Convert back to uint8
    output = uint8(output);
end

%% Q1.6 Apply custom median filters
filtered_median_3 = my_median_filter(img1_gray, 3);
filtered_median_5 = my_median_filter(img1_gray, 5);

% Save median filtered images
if ~exist('./L2/outputs/01_smoothing/06', 'dir')
    mkdir('./L2/outputs/01_smoothing/06');
end
imwrite(filtered_median_3, './L2/outputs/01_smoothing/06/median3.png');
imwrite(filtered_median_5, './L2/outputs/01_smoothing/06/median5.png');

% Step 6: Show all results in one subplot figure
figure;
subplot(2, 4, 1); imshow(img1_gray); title('Original Grayscale');
subplot(2, 4, 2); imshow(filtered1); title('(1/9) ones(3)');
subplot(2, 4, 3); imshow(filtered2); title('(1/49) ones(7)');
subplot(2, 4, 4); imshow(filtered3); title('average (7x7)');
subplot(2, 4, 5); imshow(filtered4); title('gaussian (3x3, 0.5)');
subplot(2, 4, 6); imshow(filtered5); title('gaussian (7x7, 1.2)');
subplot(2, 4, 7); imshow(filtered_median_3); title('Median (3x3)');
subplot(2, 4, 8); imshow(filtered_median_5); title('Median (5x5)');

%% Q2: Sharpening and Edge Detection
% Read img2 and convert to grayscale
img2 = imread('./L2/img2.png');
img2_gray = rgb2gray(img2);

% Create output directory for sharpening
if ~exist('./L2/outputs/02_sharpening', 'dir')
    mkdir('./L2/outputs/02_sharpening');
end

% Save the grayscale image
imwrite(img2_gray, './L2/outputs/02_sharpening/img2_grey.png');

% Define sharpening kernels
kernel6 = [-1, 0, 1; -2, 0, 2; -1, 0, 1];
kernel7 = [-1, -2, -1; 0, 0, 0; 1, 2, 1];
kernel8 = fspecial('log', 3);

% Apply sharpening filters using the custom function
filtered6 = my_filter2D(img2_gray, kernel6);
filtered7 = my_filter2D(img2_gray, kernel7);
filtered8 = my_filter2D(img2_gray, kernel8);

% Apply filters to sharpen the original image
% Convert to double for arithmetic operations and clamp results
sharpened_img6 = uint8(max(0, min(255, double(img2_gray) + double(filtered6))));
sharpened_img7 = uint8(max(0, min(255, double(img2_gray) + double(filtered7))));
```

```
sharpened_img8 = uint8(max(0, min(255, double(img2_gray) - double(filtered8))));

% Save sharpening filtered images and sharpened results
imwrite(filtered6, './L2/outputs/02_sharpening/kernel6.png');
imwrite(filtered7, './L2/outputs/02_sharpening/kernel7.png');
imwrite(filtered8, './L2/outputs/02_sharpening/kernel8.png');
imwrite(sharpened_img6, './L2/outputs/02_sharpening/sharpened6.png');
imwrite(sharpened_img7, './L2/outputs/02_sharpening/sharpened7.png');
imwrite(sharpened_img8, './L2/outputs/02_sharpening/sharpened8.png');

% Display sharpening results
figure;
subplot(3, 3, 1); imshow(img2_gray); title('Original img2 Grayscale');
subplot(3, 3, 2); imshow(filtered6); title('Kernel 6 (Vertical Edge)');
subplot(3, 3, 3); imshow(filtered7); title('Kernel 7 (Horizontal Edge)');
subplot(3, 3, 4); imshow(filtered8); title('Kernel 8 (Laplacian of Gaussian)');
subplot(3, 3, 5); imshow(sharpened_img6); title('Sharpened with Kernel 6');
subplot(3, 3, 6); imshow(sharpened_img7); title('Sharpened with Kernel 7');
subplot(3, 3, 7); imshow(sharpened_img8); title('Sharpened with Kernel 8');
```

Appendix B

This section contains the full code implemented for the Thresholding section. It is identical to the submitted file `thresholding.m`.

```
% ECE 7410 - Laboratory 2: Thresholding

%% Q2: Function to calculate optimal threshold using Otsu's method
function optimal_threshold = calculate_optimal_threshold(img)
    % Convert to grayscale if needed
    if size(img, 3) == 3
        img = rgb2gray(img);
    end

    % Convert to double for calculations
    img = double(img);

    % Calculate histogram
    [counts, ~] = hist(img(:), 0:255);

    % Calculate probabilities
    p = counts / sum(counts);

    % Initialize variables
    L = 256; % Number of gray levels (0-255)
    max_variance = 0;
    optimal_threshold = 0;

    % Try all possible thresholds
    for t = 1:(L - 1)
        % Calculate class probabilities
        omega_0 = sum(p(1:t));
        omega_1 = sum(p(t + 1:L));
```

```
        % Skip if one class is empty
        if omega_0 == 0 || omega_1 == 0
            continue;
        end

        % Calculate class means
        mu_0 = sum((0:(t - 1)) .* p(1:t)) / omega_0;
        mu_1 = sum((t:(L - 1)) .* p(t + 1:L)) / omega_1;

        % Calculate inter-class variance
        sigma_b_squared = omega_0 * omega_1 * (mu_0 - mu_1) ^ 2;

        % Update optimal threshold if this variance is larger
        if sigma_b_squared > max_variance
            max_variance = sigma_b_squared;
            optimal_threshold = t - 1; % Convert to 0-255 range
        end
    end
end

%% Q3: Calculate optimal thresholds for test images
% Read images
img3 = imread('./L2/img3.png');
img4 = imread('./L2/img4.png');

% Calculate optimal thresholds
threshold_img3 = calculate_optimal_threshold(img3);
threshold_img4 = calculate_optimal_threshold(img4);

% Display results
fprintf('Optimal threshold for img3.png: %d\n', threshold_img3);
fprintf('Optimal threshold for img4.png: %d\n', threshold_img4);

%% Q4: Function to convert grayscale to binary using threshold
function binary_img = grayscale_to_binary(img, threshold)
    % Convert to grayscale if needed
    if size(img, 3) == 3
        img = rgb2gray(img);
    end

    % Convert to double for calculations
    img = double(img);

    % Get image dimensions
    [rows, cols] = size(img);

    % Initialize binary image
    binary_img = zeros(rows, cols);

    % Apply threshold manually using loops
    for i = 1:rows
        for j = 1:cols
            if img(i, j) > threshold
```

```
                binary_img(i, j) = 255; % White pixel
            else
                binary_img(i, j) = 0; % Black pixel
            end
        end
    end

    % Convert to uint8
    binary_img = uint8(binary_img);
end

%% Q5: Generate and save binary images using optimal thresholds
% Create output directory
if ~exist('./L2/outputs/03_thresholding', 'dir')
    mkdir('./L2/outputs/03_thresholding');
end

% Convert to binary using optimal thresholds
binary_img3 = grayscale_to_binary(img3, threshold_img3);
binary_img4 = grayscale_to_binary(img4, threshold_img4);

% Display and save results
figure('Position', [100, 100, 1200, 400]);

% Display img3 results
subplot(2, 3, 1);
imshow(img3);
title('Original img3.png');

subplot(2, 3, 2);
if size(img3, 3) == 3
    imshow(rgb2gray(img3));
else
    imshow(img3);
end
title('Grayscale img3.png');

subplot(2, 3, 3);
imshow(binary_img3);
title(sprintf('Binary img3 (threshold = %d)', threshold_img3));

% Display img4 results
subplot(2, 3, 4);
imshow(img4);
title('Original img4.png');

subplot(2, 3, 5);
if size(img4, 3) == 3
    imshow(rgb2gray(img4));
else
    imshow(img4);
end
title('Grayscale img4.png');
```

```
subplot(2, 3, 6);
imshow(binary_img4);
title(sprintf('Binary img4 (threshold = %d)', threshold_img4));

% Save binary images
imwrite(binary_img3, './L2/outputs/03_thresholding/bin3.png');
imwrite(binary_img4, './L2/outputs/03_thresholding/bin4.png');

% Save the figure
saveas(gcf, './L2/outputs/03_thresholding/thresholding_results.png');
fprintf('Binary images saved to ./outputs/03_thresholding/\n');
```